Python OOP — Object-Oriented Programming

OOPچیست؟

OOPمخفف **Object-Oriented Programming**

به فارسی: **برنامه‌نویسی شیءگرا**

در OOP برنامه را بر اساس **() Objects**و **() Classes**طراحی می‌کنیم

مثلاً در یک برنامه مدرسه می‌توانیم دانش‌آموز را به صورت یک Object در نظر بگیریم:

Classچیست؟

Classیک الگو یا قالب برای ساخت Object است

مثلاً:

```
class Student:
    pass
```

اینجا یک کلاس به نام Studentساخته‌ایم

هنوز دانش‌آموزی ایجاد نکرده‌ایم؛ فقط قالب دانش‌آموز را تعریف کرده‌ایم

می‌توانیم آن را بصورت زیر تصور کنیم:

Class

↓

Student Template

↓

name

age

student_id

...

Objectچیست؟

وقتی از یک Class یک نمونه ایجاد می‌کنیم، یک **Object**داریم

```
class Student:
    pass
student1 = Student()
student2 = Student()
```

اینجا:

student1و student2دو Object متفاوت هستند که از کلاس Studentساخته شده‌اند

یک مثال ساده با Car

فرض کنید می‌خواهیم برای ماشین‌ها یک Class بسازیم:

```
class Car:
    pass
```

حالا Object ایجاد می‌کنیم:

```
car1 = Car()
car2 = Car()
```

می‌توانیم بگوییم:

Class → Car

Objects:

car1 → یک ماشین

car2 → یک ماشین دیگر

Attributesچیست؟

می‌توانیم برای Object اطلاعاتی ذخیره کنیم

این اطلاعات را **Attribute**می‌نامیم

مثلاً:

```
class Car:
    pass
car1 = Car()
car1.brand = "Toyota"
car1.color = "Red"
car1.year = 2024
```

حالا:

```
print(car1.brand)
print(car1.color)
print(car1.year)
```

خروجی:

```
Toyota
Red
2024
```

مشکل این روش چیست؟

اگر بخواهیم چندین Object ایجاد کنیم، مجبور می‌شویم اطلاعات را برای هر Object جداگانه تنظیم کنیم:

```
car1 = Car()
car1.brand = "Toyota"
car1.color = "Red"
car2 = Car()
car2.brand = "BMW"
car2.color = "Black"
car3 = Car()
car3.brand = "Audi"
car3.color = "White"
```

این روش چندان مناسب نیست اینجاست که ()__init__وارد می‌شود

متد ()__init__

()__init__یک متد ویژه است که هنگام ایجاد Object اجرا می‌شود

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
```

حالا:

```
car1 = Car("Toyota", "Red")
car2 = Car("BMW", "Black")
car3 = Car("Audi", "White")
```

می‌توانیم اطلاعات آنها را چاپ کنیم:

```
print(car1.brand)
print(car2.brand)
print(car3.brand)
```

خروجی:

```
Toyota
BMW
Audi
```

selfچیست؟

یکی از مهم‌ترین مفاهیم OOP درPython ، selfاست

در این کد:

```
class Car:
```

```
  def __init__(self, brand, color):
    self.brand = brand
    self.color = color
```

self به Objectفعلی اشاره می‌کند

مثلاً:

```
car1 = Car("Toyota", "Red")
```

تقریباً می‌توانیم تصور کنیم:

```
self → car1
```

و وقتی:

```
car2 = Car("BMW", "Black")
```

اجرا می‌شود:

```
self → car2
```

بنابراین:

```
self.brand = brand
```

یعنی:

مقدار brandرا در Attribute مربوط به همین Object ذخیره کن

تفاوت brandو self.brand

این موضوع برای دانشجویان معمولاً کمی گیج‌کننده است.

```
class Car:
  def __init__(self, brand):
    self.brand = brand
```

سمت راست brand پارامتر تابع است و سمت چپ self.brand ، Attributeمربوط به Object است

مثلاً:

```
car1 = Car("Toyota")
```

نتیجه:

brand → "Toyota"

self.brand → "Toyota"

Methodsچیست؟

تابعی که داخل Class قرار دارد، Methodنامیده می‌شود

مثلاً:

```
class Car:
    def start(self):
        print("Car started")
```

حالا:

```
car1 = Car()
car1.start()
```

خروجی:

Car started

Methodبا Attribute

می‌توانیم از Attribute داخل Method استفاده کنیم:

```
class Car:
    def __init__(self, brand):
        self.brand = brand
    def show_brand(self):
        print("Brand:", self.brand)
```

حالا:

```
car1 = Car("Toyota")
car2 = Car("BMW")
```

```
car1.show_brand()
car2.show_brand()
```

خروجی:

```
Brand: Toyota
Brand: BMW
```

یک مثال کامل

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print("My name is", self.name)
        print("I am", self.age, "years old")
student1 = Student("Ali", 20)
student2 = Student("Sara", 22)
student1.introduce()
student2.introduce()
```

خروجی:

```
My name is Ali
I am 20 years old
My name is Sara
I am 22 years old
```

Classو Object در دنیای واقعی

مثال :

Class

Car

Objects

Toyota

BMW

Audi

Mercedes

هر Object می‌تواند اطلاعات متفاوتی داشته باشد:

car1

brand = Toyota

color = Red

year = 2024

car2

brand = BMW

color = Black

year = 2023

اما هر دو از یک Class ساخته شده‌اند:

چرا از OOP استفاده می‌کنیم؟

فرض کنید یک سیستم مدیریت دانشگاه داریم بدون OOP ممکن است متغیرهای زیادی داشته باشیم:

student1_name = "Ali"

student1_age = 25

```
student2_name = "Sara"
student2_age = 22
student3_name = "John"
student3_age = 24
```

اما با:OOP

```
student1 = Student("Ali", 25)
student2 = Student("Sara", 22)
student3 = Student("John", 24)
```

کد مرتب‌تر و قابل توسعه‌تر می‌شود

مزایای OOP

سازمان‌دهی بهتر

کد مربوط به یک موجودیت در یک Class قرار می‌گیرد

Reusability

یک Class را می‌توانیم بارها استفاده کنیم

کاهش تکرار کد

به اصل **DRY** کمک می‌کند:

Don't Repeat Yourself

نگهداری آسان‌تر

در پروژه‌های بزرگ، تغییر دادن کد راحت‌تر می‌شود

مناسب پروژه‌های بزرگ

OOP در پروژه‌هایی مانند سیستم‌های بانکی ، فروشگاه‌های اینترنتی ، سیستم‌های دانشگاهی ، بازی‌ها ، برنامه‌های دسکتاپ ، وب‌سایت‌ها ، پروژه‌های AI و Machine Learning

تمرین ۱ - کلاس Student

یک Class به نام Student ایجاد کنید

برای هر دانشجو موارد زیر را ذخیره کنید:

name

age

major

سپس دو Student ایجاد کنید و اطلاعات آنها را چاپ کنید

تمرین ۲ - کلاسCar

یک Class به نام Carایجاد کنید که دارای:

brand

model

year

باشد

مثلاً:

car1 = Car("Toyota", "Corolla", 2023)

و اطلاعات ماشین را چاپ کنید

تمرین ۳Method

کلاس Carرا طوری طراحی کنید که یک Method به نام ()startداشته باشد

خروجی:

Toyota Corolla is starting...

تمرین ۴ Bank Account

یک Class به نام BankAccountایجاد کنید

Attributes:

owner

balance

Methods:

deposit()

withdraw()

show_balance()

مثلاً:

```
account = BankAccount("Ali", 1000)
account.deposit(500)
account.withdraw(200)
account.show_balance()
```

خروجی:

```
Balance: 1300
```

تمرین ۵Rectangle

یک Class به نام Rectangleایجاد کنید

Attributes:

width

height

وMethod های زیر را ایجاد کنید:

area()

perimeter()

مثلاً:

```
rectangle = Rectangle(10, 5)
print(rectangle.area())
print(rectangle.perimeter())
```

خروجی:

50

30

تمرین ۶Employee

یک Class به نام Employeeبسازید:

name

salary

department

و یک Method به نام show_info()ایجاد کنید

خروجی مثلاً:

Name: Ali

Salary: 2500

Department: IT

تمرین ۷ - پروژه

یک سیستم ساده‌ی Libraryطراحی کنید

Class:

Book

Attributes:

title

author

year

Methods:

show_info()

سپس حداقل سه کتاب ایجاد کنید و اطلاعات آنها را نمایش دهید

Classچیست؟

Classیعنی یک قالب یا نقشه (Blueprint) برای ساختنObject ها ، مثلاً فرض کنید می‌خواهیم اطلاعات چند دانش‌آموز را در برنامه نگهداری کنیم.

به‌جای اینکه برای هر دانش‌آموز متغیرهای جداگانه بنویسیم:

```
student1_name = "Ali"
student1_age = 20
student2_name = "Sara"
student2_age = 22
```

می‌توانیم یک classبسازیم:

```
class Student:
    pass
```

حالا Studentیک **Class**است

Objectچیست؟

وقتی از روی یک Class یک نمونه می‌سازیم، آن نمونه را Objectمی‌گوییم

```
class Student:
    pass
student1 = Student()
student2 = Student()
```

ساخت یکClass

برای ساخت Class از کلمه classاستفاده می‌کنیم:

```
class MyClass:
    x = 5
```

اینجا MyClass نام Class است و x = 5 یک **Attribute / Property**مربوط به Class است

ساختObject

حالا می‌توانیم از MyClassیک Object بسازیم:

```
class MyClass:
    x = 5
```

```
p1 = MyClass()
print(p1.x)
```

خروجی:

```
5
```

این خط:

```
p1 = MyClass()
```

یعنی یک Object از Class MyClass بساز و آن را در متغیر p1قرار بده

چند Object از یکClass

یکی از مزیت‌های مهم Class این است که می‌توانیم چند Object از یک Class بسازیم:

```
class MyClass:
  x = 5
p1 = MyClass()
p2 = MyClass()
p3 = MyClass()
print(p1.x)
print(p2.x)
print(p3.x)
```

خروجی:

```
5
5
5
```

یعنی یک Class می‌تواند برای ساخت تعداد زیادی Object استفاده شود

مثال واقعی‌ترStudent :

حالا بهتر است مثال را از MyClassبه یک مثال واقعی تبدیل کنیم:

```
class Student:
    name = "Ali"
    age = 20
```

Objectمی‌سازیم:

```
student1 = Student()
print(student1.name)
print(student1.age)
```

خروجی:

```
Ali
20
```

نکته بسیار مهم Class :و Object را اشتباه نکنید

مثلاً:

```
class Car:
    brand = "Toyota"
```

اینجا:

Car → Class

اما:

```
car1 = Car()
car2 = Car()
```

اینجا:

car1 → Object

car2 → Object

آیاObject ها مستقل هستند؟

بله، Objectهای مختلف از یک Class می‌توانند مستقل از یکدیگر باشند؛ اما باید در این مثال مقدماتی یک نکته را اصلاح کنیم:

این جمله W3Schools که می‌گوید هر» Object کپی خودش ازproperty های «Class را دارد، برای این مثال دقیق نیست.

در:

```
class MyClass:
  x = 5
```

xدر واقع **Class Attribute**است و در سطح Class تعریف شده است

اگر بنویسیم:

```
p1 = MyClass()
p2 = MyClass()
p1.x = 10
print(p1.x)
print(p2.x)
```

خروجی:

```
10
5
```

چرا؟ چون با p1.x = 10 یک attribute مخصوص p1ایجاد شده است

حذفObject

می‌توانیم با delیک Object را حذف کنیم:

```
class Person:
  pass
p1 = Person()
del p1
```

بعد از این:

print(p1) خطا خواهیم داشت، چون p1دیگر وجود ندارد

passدرClass

اگر فعلاً نمی‌خواهیم داخل Class چیزی بنویسیم:

```
class Person:
   pass
```

از passاستفاده می‌کنیم

بدون :pass

```
class Person:
```

کد ناقص است و خطا ایجاد می‌کند

معنی ساده	مفهوم
ساختن یک قالب	class
نقشه/قالب برایObject ها	Class
نمونه‌ای از یکClass	Object
ویژگی Object یاClass	Property / Attribute
ساختن یکObject	Class()
حذف یکObject	del
خالی گذاشتن Class به‌صورت موقت	pass

مثال

```
class Student:
   name = "Ali"
   age = 20
student1 = Student()
print(student1.name)
print(student1.age)
```

Python __init__() Method

__init__()چیست؟

__init__()یک متد ویژه (Special Method) در Python است که هنگام ساختن یک Object اجرا می‌شود

به زبان ساده : __init__()برای مقداردهی اولیه Object استفاده می‌شود

مثلاً اگر بخواهیم برای هر دانش‌آموز nameو ageداشته باشیم:

```
class Student:
   def __init__(self, name, age):
      self.name = name
      self.age = age
```

حالا:

```
student1 = Student("Ali", 20)
```

هنگام ساخت student1، پایتون به‌صورت خودکار __init__()را اجرا می‌کند

اتفاقی که پشت صحنه می‌افتد

وقتی می‌نویسیم:

```
student1 = Student("Ali", 20)
```

پایتون تقریباً این کار را انجام می‌دهد:

Student("Ali", 20)

↓

__init__(self, name, age)

↓

name = "Ali"

age = 20

↓

self.name = "Ali"

self.age = 20

در نتیجه student1.name برابر است با: Ali

و student1.age برابر است با ۲۰

selfچیست؟

در:def __init__(self, name, age) سه پارامتر داریم: ، self ، name ، age

اما هنگام ساخت:Object

```
student1 = Student("Ali", 20)
```

ما فقط دو مقدار می‌دهیم:

"Ali"

20

چون selfرا خود Python به‌صورت خودکار ارسال می‌کند

پس: self به Object فعلی اشاره می‌کند.

در این مثال:

```
student1 = Student("Ali", 20)
```

می‌توان تقریباً تصور کرد:

self → student1

بنابراین:

```
self.name = name
```

یعنی مقدار nameرا داخل ویژگی nameمربوط به همین Object قرار بده

تفاوت nameو self.name

این یکی از مهم‌ترین نکات برای دانش‌آموزان است

در:

```
def __init__(self, name, age):
    self.name = name
    self.age = age
```

دو nameداریم: name یک پارامتر محلی است اما self.name یک Attributeمتعلق به Object است

مثلاً:

```
student1 = Student("Ali", 20)
```

در این لحظه:

name

↓

"Ali"

self.name

↓

student1.name

↓

"Ali"

پس: self.name = name یعنی Attribute Object = Parameter

ساخت چندObject

قدرت واقعی __init__()اینجا مشخص می‌شود

```
class Student:
   def __init__(self, name, age):
      self.name = name
      self.age = age
```

حالا:

```
student1 = Student("Ali", 20)
student2 = Student("Sara", 22)
student3 = Student("John", 19)
```

مثلاً:

```
print(student1.name)
```

```
print(student2.name)
print(student3.name)
```

خروجی:

```
Ali
Sara
John
```

چرا __init__()__مهم است؟

بدون __init__()__مجبوریم اطلاعات را بعد از ساخت Object وارد کنیم:

```
class Student:
    pass
student1 = Student()
student1.name = "Ali"
student1.age = 20
```

برای Object بعدی:

```
student2 = Student()
student2.name = "Sara"
student2.age = 22
```

این روش طولانی و مستعد خطاست

اما با __init__():

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

فقط می‌نویسیم:

```
student1 = Student("Ali", 20)
student2 = Student("Sara", 22)
```

مثال :

برای تدریس می‌توانید از Carاستفاده کنید:

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand
        self.model = model
        self.year = year
car1 = Car("Toyota", "Corolla", 2020)
car2 = Car("BMW", "X5", 2023)
print(car1.brand)
print(car1.model)
print(car1.year)
print(car2.brand)
print(car2.model)
print(car2.year)
```

هر Object اطلاعات خودش را دارد

()__init__فقط برای ذخیره اطلاعات نیست

می‌توانیم هنگام ایجاد Object کارهای دیگری هم انجام دهیم

مثلاً:

```
class Person:
    def __init__(self, name, age):
        self.name = name
```

```
        self.age = age
        print("A new person was created!")
```

وقتی بنویسیم:

```
p1 = Person("Ali", 43)
```

خروجی:

```
A new person was created!
```

چون ()__init__به‌صورت خودکار اجرا شده است

یک نکته مهم درباره نام __init__

این:

__init__

دو underscore در ابتدا و دو underscore در انتها دارد به این نوع متدها معمولاً **Dunder Methods**یا **Special Methods**گفته می‌شود initمخفف **initialize**است؛ یعنی مقداردهی اولیه

مثال :

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
s1 = Student("Ali", 20)
print(s1.name)
print(s1.age)
```

Python self Parameter

selfچیست؟

selfیعنی همین **Object** فعلی

مثلاً:

```
class Person:
   def __init__(self, name):
      self.name = name
```

اگر بنویسیم:

```
p1 = Person("Ali")
```

در اینجا selfبه p1اشاره می‌کند

اگر بنویسیم:

```
p2 = Person("Sara")
```

در اینجا selfبه p2اشاره می‌کند

چرا به selfنیاز داریم؟

فرض کنید:

```
class Person:
   def __init__(self, name):
      self.name = name
   def printname(self):
      print(self.name)
```

حالا:

```
p1 = Person("Ali")
p2 = Person("Sara")
p1.printname()
p2.printname()
```

خروجی:

```
Ali
Sara
```

چرا؟وقتی می‌نویسیم p1.printname() ایتون می‌فهمد که selfمربوط به p1است

پس: self.nameدر اینجا یعنیp1.nameو وقتی()2.printnameرا اجرا می‌کنیم:

self.nameیعنیp2.name

self.nameدقیقاً چیست؟

در:

```
def __init__(self, name):
    self.name = name
```

دو nameداریم:

name

↓

پارامتر

self.name

↓

Attributeمربوط بهObject

مثلاً:

```
p1 = Person("Ali")
```

در نتیجه:

```
name      = "Ali"
self.name  = p1.name = "Ali"
```

پس:

```
self.name = name
```

یعنی:

مقدار پارامتر nameرا در Attribute مربوط به Object فعلی ذخیره کن

selfدر متدها

selfفقط برای ()__init__نیست

مثلاً:

```
class Person:
   def __init__(self, name):
      self.name = name
   def greet(self):
      print("Hello, my name is " + self.name)
```

حالا:

```
p1 = Person("Ali")
p1.greet()
```

خروجی:

```
Hello, my name is Ali
```

چون:

```
self.name
```

به:

```
p1.name
```

اشاره می‌کند

selfاولین Parameter متد است ر متدهای Instance معمولاً selfبه‌عنوان اولین پارامتر نوشته می‌شود:

```
class Person:
   def greet(self):
      print("Hello")
```

یا:

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.age = age
```

اما هنگام فراخوانی، selfرا خودمان نمی‌نویسیم:

```
p1 = Person("Ali", 20)
p1.greet()
```

این کار معمولاً درست نیست:

```
p1.greet(p1)
```

چون Python خودش Object را به متد Instance می‌دهد

یک نکته بسیار مهم درباره self

این()p1.greetرا می‌توان از نظر مفهومی تقریباً این‌گونه تصور کرد:

```
Person.greet(p1)
```

یعنی p1 Python Object را به‌عنوان selfبه متد می‌دهد

پس:

```
p1.greet()
```

تقریباً:

```
p1 → self
```

است

آیا حتماً باید اسم آن selfباشد؟

از نظر فنی، خیرمثلاً این کد از نظر Python معتبر است:

```
class Person:
```

```
  def __init__(myobject, name):
    myobject.name = name
  def greet(abc):
    print("Hello", abc.name)
```

اما نباید این کار را انجام دهید ستاندارد و Convention پایتون این است که از:

Self ستفاده کنیم نابراین:

def greet(self):

بهتر و خواناتر از:

def greet(abc):

می یاشد

دسترسی به چند Attribute با self

مثلاً یک ماشین:

```
class Car:
  def __init__(self, brand, model, year):
    self.brand = brand
    self.model = model
    self.year = year
  def display_info(self):
    print(f"{self.year} {self.brand} {self.model}")
```

ساخت:Object

```
car1 = Car("Toyota", "Corolla", 2020)
```

فراخوانی:

```
car1.display_info()
```

خروجی:

```
2020 Toyota Corolla
```

اینجا:

self.brand

self.model

self.year

همگی متعلق به car1هستند

selfمی‌تواند متد دیگری را هم صدا بزند

این قسمت کمی پیشرفته‌تر است

```
class Person:
   def __init__(self, name):
      self.name = name
   def greet(self):
      return "Hello, " + self.name
   def welcome(self):
      message = self.greet()
      print(message + "! Welcome.")
```

حالا:

```
p1 = Person("Ali")
p1.welcome()
```

خروجی:

```
Hello, Ali! Welcome
```

در:

```
self.greet()
```

یعنی:متد ()greetرا برای همین Object اجرا کن

اگر selfمربوط به p1باشدself.greet()تقریباً یعنی:

```
p1.greet()
```

ارتباط class، object، __init__و self

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print("My name is", self.name)
        print("I am", self.age, "years old")
s1 = Student("Ali", 20)
s1.introduce()
```

اینجا:

مفهوم	کد
Class	class Student
ساختObject	s1 = Student(...)
مقداردهی اولیه	__init__()
اشاره به Object فعلی	self
Attributeمربوط بهObject	self.name
Method	introduce()
دسترسی به Attribute همانObject	self.nameداخل introduce()

self به Object فعلی اشاره می‌کند و با استفاده از آن می‌توانیم به Attribute ها و Method های همان Object دسترسی داشته باشیم.

Python Class Properties

Propertyچیست؟

Propertyیعنی ویژگی یا داده‌ای که به یک Object یا Class مربوط است

مثلاً یک شخص می‌تواند ویژگی‌های زیر را داشته باشد:

Person

name

age

city

در Python معمولاً این ویژگی‌ها را به شکل Attribute می‌بینیم:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

اینجا:

self.name

self.age

دو Instance Propertyهستند

ساختProperty

مثلاً:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

حالا:

```
p1 = Person("Ali", 20)
```

در نتیجه Object p1 دارای این اطلاعات است:

p1

name → "Ali"

age → 20

دسترسی به Property

برای دسترسی از **Dot Notation (.)** استفاده می‌کنیم:

```
print(p1.name)
print(p1.age)
```

خروجی:

```
Ali
20
```

یعنی:

```
p1.name
```

یعنی:

Property name مربوط به Object p1

تغییر Property

Propertyهای Object را می‌توان تغییر داد

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p1 = Person("Ali", 20)
print(p1.age)
p1.age = 21
print(p1.age)
```

خروجی:

```
20
21
```

یعنی:

```
p1.age = 21
```

مقدار ageمربوط به p1را تغییر می‌دهد

حذفProperty

با delمی‌توان یک Property را حذف کرد:

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.age = age
p1 = Person("Ali", 20)
del p1.age
print(p1.name)
```

اما:

```
print(p1.age)
```

خطا ایجاد می‌کند، چون ageاز p1حذف شده است

مهم‌ترین قسمت Instance Property :در مقابلClass Property

تفاوت دو مفهوم :

Instance Property

اگر Property داخل ()__init__و با selfتعریف شود:

```
class Person:
   def __init__(self, name):
      self.name = name
```

این Property مربوط به هر Object به‌صورت جداگانه است

مثلاً:

```
p1 = Person("Ali")
p2 = Person("Sara")
```

داریم:

p1

name → Ali

p2

name → Sara

پس:

p1.name

و:

p2.name

می‌توانند متفاوت باشند

Class Propertyچیست؟

اگر Property مستقیماً داخل Class و خارج از متدها تعریف شود:

```
class Person:
    species = "Human"
```

این یک Class Property / Class Attributeاست

مثلاً:

```
class Person:
    species = "Human"
    def __init__(self, name):
        self.name = name
```

حالا:

```
p1 = Person("Ali")
p2 = Person("Sara")
```

هر دو می‌توانند به:

p1.species

p2.species

دسترسی داشته باشند

خروجی:

Human

Human

چون speciesدر Class تعریف شده است

مثال کامل

```
class Person:
   species = "Human"      # Class Property
   def __init__(self, name, age):
      self.name = name    # Instance Property
      self.age = age      # Instance Property
p1 = Person("Ali", 20)
p2 = Person("Sara", 22)
print(p1.name)
print(p1.age)
print(p2.name)
print(p2.age)
print(p1.species)
print(p2.species)
```

خروجی:

Ali

20

Sara

22

Human

Human

تغییر Class Property

می‌توانیم Class Property را از طریق خود Class تغییر دهیم:

```
class Person:
   species = "Human"
   def __init__(self, name):
      self.name = name
p1 = Person("Ali")
p2 = Person("Sara")
Person.species = "Homo sapiens"
print(p1.species)
print(p2.species)
```

خروجی:

Homo sapiens

Homo sapiens

چون هر دو Object از Class Property استفاده می‌کنند

یک نکته بسیار مهم درباره تغییر Class Property

```
class Person:
   species = "Human"
```

و:

```
p1 = Person()
p2 = Person()
```

اگر بنویسیم:

```
Person.species = "Homo sapiens"
```

هر دو Object مقدار جدید را می‌بینند:

```
print(p1.species)
print(p2.species)
```

اما اگر بنویسیم:

```
p1.species = "Robot"
```

دیگر speciesمخصوص p1ایجاد می‌شود

یعنی:

Person.species

↓

"Homo sapiens"

p1.species

↓

"Robot"

p2.species

↓

"Homo sapiens"

Class Attributeیک مقدار مشترک در سطح Class است؛ اما اگر با selfمقداردهی کنیم، Attributeمربوط به همان Instance خواهد بود

اضافه کردن Property جدید بهObject

Pythonاجازه می‌دهد بعداً Property جدیدی به یک Object اضافه کنیم:

```
class Person:
```

```
    def __init__(self, name):
        self.name = name
p1 = Person("Ali")
p1.age = 20
p1.city = "Baku"
print(p1.name)
print(p1.age)
print(p1.city)
```

اما این ویژگی‌ها فقط برای p1ایجاد شده‌اند

اگرp2 = Person("Sara")بنویسیم:

print(p2.age)خطا می‌گیریم، چون ageبرای p2ایجاد نشده است

نوع	محل تعریف	متعلق به
Instance Property	داخل متد، معمولاً __init__()	هرObject
Class Property	مستقیماً داخلClass	Classو به‌صورت پیش‌فرض قابل دسترسی برایObject ها

مثال:

```
class Person:
    species = "Human"         # Class Property
    def __init__(self, name):
        self.name = name      # Instance Property
```

اگر ویژگی با selfبرای یک Object ساخته شود، Instance Propertyاست؛ اگر مستقیماً داخل Class تعریف شود، Class Propertyاست.

مفهوم Inheritance چیست؟

Inheritanceیعنی یک کلاس جدید بتواند ویژگی‌ها و رفتارهای یک کلاس موجود را به ارث ببرد

مثلاً:

```
class Person:
```

```
    def __init__(self, name):
        self.name = name
    def greet(self):
        print("Hello", self.name)
```

حالا می‌توانیم یک کلاس Studentبسازیم که از Personارث‌بری کند:

```
class Student(Person):
    pass
```

اکنون Student، ویژگی‌ها و متدهای Personرا دارد:

```
student1 = Student("Ali")
print(student1.name)
student1.greet()
```

خروجی:

```
Ali
Hello Ali
```

Parentو Child Class

دو اصطلاح مهم:

معنی	اصطلاح
کلاس والد	Parent Class
کلاس فرزند	Child Class
کلاس پایه	Base Class
کلاس مشتق‌شده	Derived Class

مثلاً:

```
class Person:
    pass
class Student(Person):
    pass
```

اینجا:

Person → Parent

Student → Child

چرا Inheritance استفاده می‌کنیم؟

فرض کنید:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print(self.name, self.age)
```

حالا اگر بخواهیم Teacherهم داشته باشیم، ممکن است دوباره همین کدها را بنویسیم

اما با:Inheritance

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print(self.name, self.age)
class Student(Person):
    pass
class Teacher(Person):
    pass
```

هم Studentو هم Teacherامکانات Personرا به ارث می‌برند

این دقیقاً با اصل DRY – Don't Repeat Yourselfمرتبط است

مشکل مهم هنگام استفاده از __init__

فرض کنید:

```
class Person:
   def __init__(self, name):
      self.name = name
class Student(Person):
   def __init__(self, name, grade):
      self.grade = grade
```

حالا:

```
student = Student("Ali", 10)
```

اینجا __init__().Personبه صورت خودکار اجرا نمی‌شود؛ چون Studentیک __init__()خودش تعریف کرده است.

در نتیجه self.nameساخته نمی‌شود

استفاده از super()

راه استاندارد برای استفاده از __init__()والد:

```
class Person:
   def __init__(self, name):
      self.name = name
class Student(Person):
   def __init__(self, name, grade):
      super().__init__(name)
      self.grade = grade
```

حالا:

```
student = Student("Ali", 10)
print(student.name)
print(student.grade)
```

خروجی:

Ali

10

super()یعنی چه؟

به زبان ساده:

super()به کلاس والد اشاره می‌کند و اجازه می‌دهد از متدهای والد استفاده کنیم

در اینجا:

```
super().__init__(name)
```

تقریباً یعنی:

```
Person.__init__(self, name)
```

اما استفاده از super()روش استانداردتر و انعطاف‌پذیرتر است

اضافه کردن Property جدید

فرزند می‌تواند علاوه بر ویژگی‌های والد، ویژگی‌های خودش را هم داشته باشد:

```
class Person:
    def __init__(self, name):
        self.name = name
class Student(Person):
    def __init__(self, name, grade):
        super().__init__(name)
        self.grade = grade
```

مثال:

```
student = Student("Ali", 18)
print(student.name)
print(student.grade)
```

اضافه کردن Method جدید

کلاس فرزند می‌تواند متد جدید هم داشته باشد:

```
class Person:
    def __init__(self, name):
        self.name = name
    def introduce(self):
        print("My name is", self.name)
class Student(Person):
    def study(self):
        print(self.name, "is studying")
```

حالا:

```
student = Student("Ali")
student.introduce()
student.study()
```

خروجی:

```
My name is Ali
Ali is studying
```

پس Studentهم متد والد را دارد:

```
introduce()
```

و هم متد خودش را:

```
study()
```

Method Overriding

یکی از مهم‌ترین قسمت‌های Inheritance همین است

فرض کنید Parent دارد:

```
class Person:
    def greet(self):
        print("Hello")
```

Childمی‌تواند متدی با همان نام ایجاد کند:

```
class Student(Person):
    def greet(self):
        print("Hello, I am a student")
```

حالا:

```
student = Student()
student.greet()
```

خروجی:

```
Hello, I am a student
```

یعنی متد فرزند جای متد والد را می‌گیرد.به این کار می‌گوییم**Method Overriding**

مثال

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print("Name:", self.name)
        print("Age:", self.age)
class Student(Person):
    def __init__(self, name, age, grade):
        super().__init__(name, age)
        self.grade = grade
```

```
    def study(self):
        print(self.name, "is studying")
    def introduce(self):
        print("Student:", self.name)
        print("Age:", self.age)
        print("Grade:", self.grade)
student1 = Student("Ali", 20, 18)
student1.introduce()
student1.study()
```

خروجی:

```
Student: Ali
Age: 20
Grade: 18
Ali is studying
```

Python Polymorphismچندریختی

کلمه‌ی **Polymorphism**از دو بخش تشکیل شده:

Poly = چند

Morph = شکل

بنابراین یعنی:

یک نام یا interface یکسان، اما رفتارهای متفاوت ثلاً متد :

()moveبرای ماشین، قایق و هواپیما می‌تواند وجود داشته باشد، اما هرکدام رفتار متفاوتی دارند:

Car → Drive

Boat → Sail

Plane → Fly

مثال

سه کلاس داریم:

```
class Car:
    def move(self):
        print("Drive")
class Boat:
    def move(self):
        print("Sail")
class Plane:
    def move(self):
        print("Fly")
```

حالا:

```
car = Car()
boat = Boat()
plane = Plane()
car.move()
boat.move()
plane.move()
```

خروجی:

```
Drive
Sail
Fly
```

نام متد در هر سه کلاس یکی است:

move()اما رفتار متفاوت است ین یعنی .**Polymorphism**

قدرت Polymorphism در Loop

```
for vehicle in (car, boat, plane):
   vehicle.move()
```

خروجی:

```
Drive
Sail
Fly
```

ما یک کد نوشته‌ایم:

```
vehicle.move()
```

اما Python بر اساس نوع object تصمیم می‌گیرد کدام move() اجرا شود

یعنی:

```
car.move()   → Car.move()
boat.move()  → Boat.move()
plane.move() → Plane.move()
```

این یکی از کاربردهای بسیار مهم Polymorphism است

Polymorphism با Inheritance

حالا مفهوم را با مبحث قبلی ترکیب کنیم یک کلاس والد داریم:

```
class Vehicle:
   def move(self):
      print("Moving")
```

و سه کلاس فرزند:

```
class Car(Vehicle):
   pass
class Boat(Vehicle):
```

```
    def move(self):
        print("Sailing")
class Plane(Vehicle):
    def move(self):
        print("Flying")
```

حالا:

```
car = Car()
boat = Boat()
plane = Plane()
```

نکته مهم اینجاست:

Carهیچ move()ندارد:

```
class Car(Vehicle):
    pass
```

بنابراین از Parent می‌گیرد:

Car.move()

↓

Vehicle.move()

اما:

Boat

خودش move()دارد:

```
def move(self):
    print("Sailing")
```

بنابراین متد Parent را overrideمی‌کند

Plane

همین کار را انجام می‌دهد:

```
def move(self):
    print("Flying")
```

حالاPolymorphism

می‌توانیم همه را در یک loop قرار دهیم:

```
for vehicle in (car, boat, plane):
    vehicle.move()
```

خروجی:

```
Moving
Sailing
Flying
```

ارتباط Inheritance وPolymorphism

Inheritance

می‌گوید:فرزند می‌تواند امکانات والد را به ارث ببرد

مثلاً:

```
class Car(Vehicle):
    pass
```

Polymorphism

می‌گوید:

اشیای مختلف می‌توانند یک متد همنام داشته باشند، ولی رفتار متفاوتی ارائه دهند.

مثلاً:

```
car.move()
boat.move()
plane.move()
```

پس:

Inheritance

↓

Reuse / Override

↓

Polymorphism

↓

Same interface → Different behavior

Function Polymorphism

Polymorphismفقط در کلاس‌ها نیست

مثلاً:

```
print(len("Hello"))
print(len([10, 20, 30]))
print(len((1, 2, 3, 4)))
```

یک تابع داریم:

len()

ولی روی انواع مختلف داده کار می‌کند

String → تعداد کاراکترها

List → تعداد عناصر

Tuple → تعداد عناصر

Dictionary → تعداد key/value ها

بنابراین len()نمونه‌ای از polymorphic behaviorاست

مثال

فرض کنید یک سیستم پرداخت داریم:

```
class CashPayment:
    def pay(self, amount):
        print("Paid", amount, "with cash")
class CardPayment:
    def pay(self, amount):
        print("Paid", amount, "with card")
class OnlinePayment:
    def pay(self, amount):
        print("Paid", amount, "online")
```

حالا:

```
payments = [
    CashPayment(),
    CardPayment(),
    OnlinePayment()
]
for payment in payments:
    payment.pay(100)
```

خروجی:

```
Paid 100 with cash
Paid 100 with card
Paid 100 online
```

ما اصلاً لازم نیست بدانیم object دقیقاً از چه کلاسی است قط می‌گوییم:

payment.pay(100)و هر object رفتار مناسب خودش را اجرا می‌کند

Encapsulationچیست؟

Encapsulationیعنی داده‌ها و توابع مرتبط با آن‌ها را داخل یک کلاس قرار دهیم و دسترسی به داده‌ها را کنترل کنیم.

مثلاً فرض کنید یک حساب بانکی داریم:

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
```

اینجا balance__یک ویژگی خصوصی است یعنی نمی‌خواهیم کاربر مستقیماً این کار را انجام دهد:

```
account.__balance = -100000
```

بلکه باید از روش‌های مشخصی برای خواندن یا تغییر آن استفاده کند

1. Public و Private وProtected

در پایتون معمولاً سه حالت را مطرح می‌کنیم:

مفهوم	دسترسی	علامت	نوع
عمومی	همه‌جا	name	Public
قراردادی	قابل دسترسی، ولی برای استفاده داخلی	_name	Protected
خصوصی	دسترسی مستقیم از بیرون توصیه/پشتیبانی نمی‌شود	__name	Private

مثلاً:

```
class Person:
    def __init__(self, name, age, salary):
        self.name = name
        self._salary = salary
        self.__age = age
```

در اینجا:

name → Public

_salary → Protected

__age → Private

Public Property

ویژگی عمومی را می‌توانیم مستقیماً از بیرون کلاس بخوانیم و تغییر دهیم:

```
class Person:
   def __init__(self, name):
      self.name = name
p1 = Person("Ali")
print(p1.name)
p1.name = "Reza"
print(p1.name)
```

خروجی:

```
Ali
Reza
```

Private Property

اگر قبل از نام ویژگی ___قرار دهیم:

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.__age = age
```

دیگر این کار:

```
p1 = Person("Ali", 25)
print(p1.__age)
```

خطا ایجاد می‌کندچرا؟چون __age یک **private attribute** است و نمی‌خواهیم مستقیماً از خارج کلاس به آن دسترسی داشته باشیم.

Getterچیست؟

اگر بخواهیم مقدار age__را از بیرون بخوانیم، یک متد برای این کار ایجاد می‌کنیم

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.__age = age
   def get_age(self):
      return self.__age
```

حالا:

```
p1 = Person("Ali", 25)
print(p1.get_age())
```

خروجی:

25

به این متد می‌گوییم:

Getterیعنی متدی که مقدار یک ویژگی را دریافت **(Get)** می‌کند

Setterچیست؟

اگر بخواهیم مقدار age__را تغییر دهیم، می‌توانیم یک Setter بسازیم:

```
class Person:
   def __init__(self, name, age):
      self.name = name
      self.__age = age
   def get_age(self):
      return self.__age
   def set_age(self, age):
      self.__age = age
```

استفاده:

```
p1 = Person("Ali", 25)
print(p1.get_age())
p1.set_age(30)
print(p1.get_age())
```

خروجی:

```
25
30
```

مزیت اصلی Setter: Validation

اینجا قدرت واقعی Encapsulation مشخص می‌شودمثلاً نمی‌خواهیم سن منفی باشد:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age
    def get_age(self):
        return self.__age
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Age must be positive")
```

حالا:

```
p1 = Person("Ali", 25)
p1.set_age(30)
print(p1.get_age())
```

```
p1.set_age(-5)
```

خروجی:

```
30
Age must be positive
```

پس:

User

↓

set_age()

↓

Validation

↓

__age

یعنی کاربر نمی‌تواند هر مقداری را وارد کند

Protectedچیست؟

اگر از یک _استفاده کنیم:

```
class Person:
   def __init__(self, salary):
      self._salary = salary
```

می‌توانیم بنویسیم:

```
p1 = Person(50000)
print(p1._salary)
```

و پایتون جلوی آن را نمی‌گیردپس _salaryواقعاً private نیست در واقع _salaryیک **Convention**است

یعنی برنامه‌نویس به دیگران می‌گوید:

این ویژگی برای استفاده داخلی کلاس در نظر گرفته شده؛ بهتر است مستقیماً به آن دست نزنید.

Private Method

حتی متدها هم می‌توانند private باشند

مثلاً:

```
class Calculator:
   def __validate(self, number):
      return isinstance(number, (int, float))
   def add(self, number):
      if self.__validate(number):
         print("Valid number")
      else:
         print("Invalid number")
```

حالا:

```
calc = Calculator()
calc.add(10)
```

ولی:

```
calc.__validate(10)
```

دسترسی مستقیم به متد خصوصی را نمی‌دهد

ایده این است که __validate() یک جزئیات داخلی implementation باشد و کاربر فقط با add() کار کند

Name Mangling

یک نکته مهم:در پایتون __age به معنای private واقعی مانند بعضی زبان‌ها نیست

Python نام آن را تغییر می‌دهد؛ به این کار می‌گوییم:

Name Mangling

مثلاً:

```
class Person:
   def __init__(self, age):
```

```
        self.__age = age
```

مثال

این مثال برای آموزش خیلی مناسب است:

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.__grade = grade
    def get_grade(self):
        return self.__grade
    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Grade must be between 0 and 100")
    def get_status(self):
        if self.__grade >= 60:
            return "Passed"
        else:
            return "Failed"
student = Student("Ali", 85)
print(student.name)
print(student.get_grade())
print(student.get_status())
student.set_grade(95)
print(student.get_grade())
print(student.get_status())
```

```
student.set_grade(150)
```

Inner Classچیست؟

Inner Classیعنی تعریف یک کلاس داخل کلاس دیگر

مثلاً:

```
class Outer:
    class Inner:
        pass
```

کلاس Innerداخل کلاس Outerقرار گرفته است

یک مثال ساده

```
class School:
    class Student:
        def say_hello(self):
            print("Hello Student")
```

برای ساختن شیء :Student

```
school = School()
student = school.Student()
student.say_hello()
```

خروجی:

```
Hello Student
```

Studentیک **Inner Class**است

چرا Inner Class استفاده می‌کنیم؟

فرض کنید یک Carداریم یک خودرو دارای موتور است:

پس می‌توانیم Engineرا داخل Carتعریف کنیم:

```
class Car:
    class Engine:
        def start(self):
            print("Engine started")
```

حالا:

```
car = Car()
engine = car.Engine()
engine.start()
```

خروجی:

```
Engine started
```

این کار باعث می‌شود کلاس‌هایی که فقط در یک context خاص کاربرد دارند، کنار هم سازمان‌دهی شوند

Inner Classخودش Object مستقل دارد

Inner Classبه این معنی نیست که Inner Object به‌صورت خودکار متعلق به Outer Object است

مثلاً:

```
class Car:
    class Engine:
        def start(self):
            print("Engine started")
```

وقتی می‌نویسیم:

```
car = Car()
```

هنوز Engine object ساخته نشده است

باید آن را بسازیم(engine = car.Engine

مثال واقعی‌تر Car :وEngine

مثال W3Schools را کمی ساده‌تر کنیم:

```
class Car:
    def __init__(self, brand):
        self.brand = brand
        self.engine = self.Engine()
    class Engine:
        def start(self):
            print("Engine started")
        def stop(self):
            print("Engine stopped")
    def drive(self):
        print(self.brand, "is driving")
car = Car("Toyota")
car.engine.start()
car.drive()
```

خروجی:

```
Engine started
Toyota is driving
```

چرا این مثال بهتر است؟

در این قسمت:

```
self.engine = self.Engine()
```

ما یک Engine object ایجاد کرده‌ایم و آن را داخل ویژگی engine مربوط به Car قرار داده‌ایم

یعنی:

car.engine به یک شیء از کلاس Engine اشاره می‌کند.

بنابراین می‌توانیم بنویسیم:

car.engine.start()

یعنی:

car

↓

engine

↓

start()

Inner Classچگونه به Outer Class دسترسی پیدا می‌کند؟

اینجا یک نکته بسیار مهم وجود دارددر پایتون، Inner Classبه‌صورت خودکار به object مربوط به Outer Class دسترسی ندارد.

مثلاً:

```
class Outer:
    def __init__(self):
        self.name = "Ali"
    class Inner:
        def show(self):
            print(self.name)
```

این کد مشکل دارد

چرا؟چون selfدر Innerبه **Inner object**اشاره می‌کند، نه .Outer object

راه‌حل

باید Outer object را به Inner object بدهیم:

```
class Outer:
    def __init__(self):
        self.name = "Ali"
```

```
    class Inner:
        def __init__(self, outer):
            self.outer = outer
        def show(self):
            print(self.outer.name)
outer = Outer()
inner = outer.Inner(outer)
inner.show()
```

خروجی:

```
Ali
```

مثال

فرض کنید یک کامپیوتر داریم:

Computer

CPU

RAM

می‌توانیم این ساختار را در Python پیاده کنیم:

```
class Computer:
    def __init__(self):
        self.cpu = self.CPU()
        self.ram = self.RAM()
    class CPU:
        def process(self):
            print("CPU is processing...")
```

```
    class RAM:
        def store(self):
            print("RAM is storing data...")
computer = Computer()
computer.cpu.process()
computer.ram.store()
```

خروجی:

```
CPU is processing...
RAM is storing data...
```

یک نکته مهم درباره Inner Class

Inner Class در Python خیلی رایج نیست برای بسیاری از پروژه‌ها بهتر است کلاس‌ها را جدا تعریف کنیم:

```
class Engine:
    def start(self):
        print("Engine started")
class Car:
    def __init__(self):
        self.engine = Engine()
```

به جای:

```
class Car:
    class Engine:
        ...
```

Inner Class زمانی مفید است که یک کلاس فقط در context یک کلاس دیگر کاربرد داشته باشد و بخواهیم ساختار کد را گروه‌بندی کنیم

معنی	مفهوم
کلاس داخل کلاس دیگر	Inner Class
کلاس بیرونی	Outer Class
شیء ساخته‌شده از Inner Class	Inner Object
ساختن Inner Object	outer.Inner()
ایجاد Engine داخل Outer Object	self.Engine()
✕وجود ندارد	دسترسی خودکار به Outer
برای دسترسی به Outer لازم است	ارسال Outer به Inner

نکته:

```
class Outer:
    class Inner:
        pass
```

یعنی:یک کلاس را داخل کلاس دیگری تعریف کرده‌ایم و اگر بخواهیم از آن object بسازیم:

```
outer = Outer()
inner = outer.Inner()
```

پس:

Class

↓

Object

↓

Inner Class

↓

Inner Object